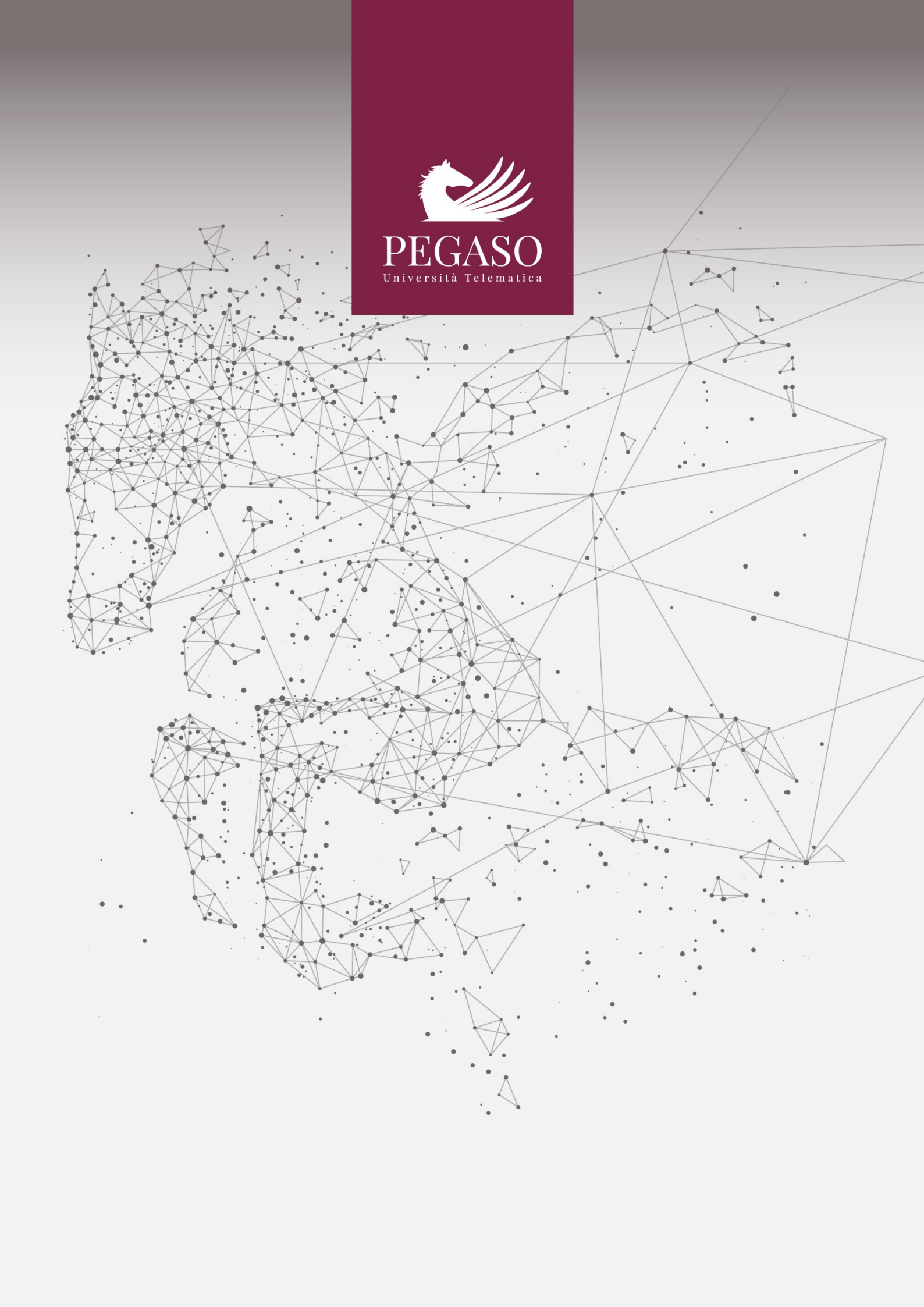

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. STATO 3
2. TODO LIST 6
3. OTTIMIZZAZIONE 9
4. CAMPO MINATO 12
BIBLIOGRAFIA 18
SITOGRAFIA 19

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Stato

Lo stato è un aspetto fondamentale della creazione di componenti dinamici in React: lo stato di un componente è un oggetto che contiene le informazioni relative al componente, come ad esempio i dati, lo stato di attivazione e altre informazioni utili per la creazione dell'interfaccia utente.

In React, lo stato di un componente può essere gestito utilizzando il metodo setState() o hooks come useState(). Il metodo setState() consente di aggiornare lo stato di un componente in modo asincrono, mentre useState() consente di gestire lo stato all'interno di un componente funzionale.

Ad esempio, il seguente componente utilizza il metodo setState() per aggiornare il contatore ogni volta che l'utente fa clic sul pulsante:

```
class Counter extends React.Component {
  constructor(props) {
    super(props);
    this.state = { count: 0 };
    this.handleClick = this.handleClick.bind(this);
  }

  handleClick() {
    this.setState({ count: this.state.count + 1 });
  }

  render() {
    return (
      <div>
        <p>You clicked {this.state.count} times</p>
        <button onClick={this.handleClick}>Click me</button>
      </div>
    );
  }
}
```

In questo esempio, il componente Counter utilizza il metodo setState() per aggiornare il valore del contatore ogni volta che l'utente fa clic sul pulsante. Il metodo setState() viene chiamato all'interno del metodo handleClick(), che viene eseguito ogni volta che l'utente fa clic sul pulsante.

Il seguente esempio utilizza useState() per gestire lo stato all'interno di un componente funzionale:

```
function Counter() {
  const [count, setCount] = useState(0);

  function handleClick() {
    setCount(count + 1);
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
  }

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={handleClick}>Click me</button>
    </div>
  );
}
```

La funzione useState() restituisce un array contenente il valore dello stato e una funzione per aggiornare lo stato. La funzione handleClick() utilizza la funzione setCount() per aggiornare il valore del contatore ogni volta che l'utente fa clic sul pulsante.

In React, è possibile creare hook personalizzati per la gestione dello stato all'interno dei componenti. Gli hook personalizzati consentono di riutilizzare la logica di gestione dello stato in diversi componenti senza doverla duplicare.

Ad esempio, il seguente hook personalizzato utilizza useState() per gestire lo stato di un componente:

```
import { useState } from 'react';

function useCounter(initialValue) {
  const [count, setCount] = useState(initialValue);

  function increment() {
    setCount(count + 1);
  }

  function decrement() {
    setCount(count - 1);
  }

  return [count, increment, decrement];
}
```

In questo esempio, l'hook personalizzato "useCounter" utilizza useState() per gestire lo stato di un componente e restituisce un array contenente il valore dello stato e le funzioni "increment" e "decrement" per aggiornare lo stato. Il valore iniziale dello stato viene passato come argomento alla funzione " useCounter ".

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Il seguente componente utilizza l'hook personalizzato "useCounter " per gestire lo stato di un contatore:

```
function Counter() {
  const [count, increment, decrement] = useCounter(0);

  return (
    <div>
      <p>You clicked {count} times</p>
      <button onClick={increment}>Increment</button>
      <button onClick={decrement}>Decrement</button>
    </div>
  );
}
```

La funzione "useCounter " restituisce un array contenente il valore dello stato e le funzioni "increment" e "decrement". Il valore dello stato viene poi utilizzato per visualizzare il numero di volte che l'utente ha fatto clic sui pulsanti "Increment" e "Decrement".

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Todo List

Di seguito un esempio in React di una Todo List:

```
import React, { useState } from 'react';

function TodoList() {
  const [todos, setTodos] = useState([]);
  const [inputValue, setInputValue] = useState('');

  const handleInputChange = (event) => {
    setInputValue(event.target.value);
  };

  const handleAddTodo = () => {
    if (inputValue.trim() !== '') {
      setTodos([...todos, inputValue.trim()]);
      setInputValue('');
    }
  };

  const handleRemoveTodo = (index) => {
    setTodos(todos.filter((_, i) => i !== index));
  };

  return (
    <div>
      <h1>Lista delle attività</h1>
      <ul>
        {todos.map((todo, index) => (
          <li key={index}>
            {todo}
            <button onClick={() => handleRemoveTodo(index)}>X</button>
          </li>
        ))}
      </ul>
      <input value={inputValue} onChange={handleInputChange} />
      <button onClick={handleAddTodo}>Aggiungi</button>
    </div>
  );
}

export default TodoList;
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

In questo esempio, utilizziamo l'hook useState di React per gestire lo stato dell'applicazione. In particolare, utilizziamo il valore todos e la funzione setTodos per gestire gli elementi della lista delle attività e il valore inputValue e la funzione setInputValue per gestire l'input dell'utente.

La funzione handleInputChange viene utilizzata per aggiornare il valore dell'input ogni volta che l'utente digita qualcosa. La funzione handleAddTodo viene utilizzata per aggiungere un nuovo elemento alla lista solo se il valore dell'input non è vuoto.

Se l'utente preme il pulsante "Aggiungi" e l'input contiene testo, il valore dell'input viene aggiunto alla lista utilizzando la sintassi dello spread operator ... e il valore dell'input viene reimpostato su una stringa vuota.

Lo spread operator ... è una funzionalità di JavaScript introdotta a partire dalla versione ES6 che consente di "spalmare" un array o un oggetto in un altro array o oggetto: in questo esempio specifico, utilizziamo lo spread operator ... per aggiungere un nuovo elemento alla lista delle attività. Ad esempio, se abbiamo un array a = [1, 2, 3] e vogliamo aggiungere l'elemento 4 alla fine dell'array, possiamo farlo utilizzando la sintassi a = [...a, 4]. Questa sintassi significa "crea un nuovo array che contiene tutti gli elementi di a, seguiti da 4".

In questo caso, stiamo creando un nuovo array che contiene tutti gli elementi della lista delle attività todos, seguiti dal valore dell'input inputValue (previa rimozione degli eventuali spazi iniziali e finali utilizzando il metodo trim()). Infine, utilizziamo la funzione setTodos per impostare il nuovo array come valore della lista delle attività.

La funzione handleRemoveTodo viene utilizzata per rimuovere un elemento dalla lista. Viene chiamata quando l'utente preme il pulsante "X" accanto a un elemento della lista e rimuove l'elemento dalla lista utilizzando il metodo filter.

Infine, mostriamo gli elementi della lista sulla pagina utilizzando un elemento <ul>, un elemento <li> per ciascun elemento della lista e un pulsante "X" accanto a ogni elemento per consentire all'utente di rimuovere l'elemento. Aggiungiamo anche un input e un pulsante per consentire all'utente di aggiungere un nuovo elemento alla lista.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).



Figura 1 *- Lista delle attività*

Il sistema di gestione delle attività può essere personalizzato in molti modi diversi, come ad esempio aggiungendo la possibilità di modificare le attività esistenti, ordinando le attività per priorità o data di scadenza, utilizzando filtri per visualizzare solo alcune attività, ecc.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Ottimizzazione

Prima di iniziare a implementare strategie di ottimizzazione, è importante analizzare le prestazioni di un'applicazione React e identificare i fattori che possono rallentare l'applicazione.

Per fare ciò, è possibile utilizzare strumenti come il profiler di React, che consente di analizzare le prestazioni di un'applicazione e di identificare i componenti che richiedono più tempo di rendering o che generano più aggiornamenti. Inoltre, è possibile utilizzare strumenti di analisi delle prestazioni come Lighthouse o WebPageTest per identificare i problemi di prestazioni a livello di rete o di rendering.

Alcuni dei fattori che possono rallentare un'applicazione React includono:

- **Rendering dei componenti**: il rendering dei componenti è un'operazione costosa in termini di prestazioni. Se un'applicazione React ha molti componenti, il tempo di rendering complessivo dell'applicazione può aumentare considerevolmente.
- **Aggiornamenti del DOM**: ogni volta che lo stato di un componente viene aggiornato, React deve aggiornare il DOM corrispondente. Questo può richiedere molto tempo, specialmente se l'applicazione ha molti componenti e il DOM è molto grande.
- **Rendering condizionale**: il rendering condizionale può rallentare un'applicazione se non viene implementato correttamente. Se un componente viene renderizzato in modo condizionale e la condizione viene verificata molte volte, il tempo di rendering del componente può aumentare considerevolmente.

Per migliorare le prestazioni di un'applicazione React, è possibile implementare diverse strategie, come:

- l'implementazione di memoization;
- la sospensione del rendering;
- l'ottimizzazione del rendering condizionale;
- […]

L'implementazione di memoization consente di memorizzare il risultato di una funzione in modo che non debba essere eseguita di nuovo se viene chiamata con gli stessi argomenti. Ciò può migliorare le prestazioni di un'applicazione React, specialmente se una funzione viene chiamata molte volte.

La sospensione del rendering consente di evitare il rendering di un componente finché non sono stati caricati tutti i dati necessari. Ciò può migliorare le prestazioni di un'applicazione React, specialmente se l'applicazione deve caricare grandi quantità di dati.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

L'ottimizzazione del rendering condizionale può migliorare le prestazioni di un'applicazione React se viene implementata correttamente. Ad esempio, è possibile utilizzare il metodo shouldComponentUpdate() per evitare il rendering di un componente se non è necessario. Inoltre, è possibile utilizzare il componente memo() di React per memorizzare il risultato del rendering di un componente e riutilizzarlo se il componente viene renderizzato di nuovo con gli stessi props.

Di seguito sono riportati alcuni esempi di codice per implementare queste strategie e migliorare le prestazioni di un'applicazione React:

```
import React, { useMemo } from 'react';

function MyComponent(props) {
  const { data } = props;

  const memoizedData = useMemo(() => {
    // Esegui il calcolo solo se i dati sono cambiati
    return calculateData(data);
  }, [data]);

  return (
    <div>
      <p>{memoizedData}</p>
    </div>
  );
}
```

In questo esempio, la funzione useMemo() viene utilizzata per memorizzare il risultato di calculateData() solo se i dati sono cambiati. In questo modo, se i dati non cambiano, la funzione calculateData() non viene eseguita di nuovo, migliorando le prestazioni dell'applicazione.

```
// Implementazione della sospensione del rendering

import React, { useState, useEffect } from 'react';

function MyComponent(props) {
  const [data, setData] = useState(null);

  useEffect(() => {
    // Carica i dati solo quando il componente viene montato
    fetchData().then((result) => {
      setData(result);
    });
  }, []);
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
  if (!data) {
    // Sospendi il rendering finché i dati non sono stati caricati
    return <div>Loading...</div>;
  }

  return (
    <div>
      <p>{data}</p>
    </div>
  );
}
```

In questo esempio, il componente utilizza useState() per gestire lo stato dei dati e useEffect() per caricare i dati solo quando il componente viene montato. Inoltre, se i dati non sono ancora stati caricati, il componente sospende il rendering e mostra un messaggio di caricamento.

```
// Implementazione dell'ottimizzazione del rendering condizionale

import React, { memo } from 'react';

function MyComponent(props) {
  const { data } = props;

  return (
    <div>
      <p>{data}</p>
    </div>
  );
}

// Memoizza il componente solo se i props sono cambiati
export default memo(MyComponent);
```

In questo esempio, il componente MyComponent viene memoizzato utilizzando il componente memo() di React. In questo modo, se il componente viene renderizzato di nuovo con gli stessi props, il risultato del rendering viene riutilizzato senza eseguire di nuovo il rendering del componente.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Campo minato

Il gioco del campo minato è un gioco di logica in cui l'obiettivo è quello di scoprire tutte le caselle del tabellone che non contengono una mina. Il tabellone è costituito da una griglia di celle, alcune delle quali contengono una mina.

Il gioco inizia con tutte le caselle coperte, ovvero l'utente non può vedere quale casella contiene una mina e quale no. L'utente può scoprire le caselle cliccandoci sopra. Se la casella scoperta contiene una mina, il gioco è perso. Se la casella scoperta non contiene una mina, verrà mostrato il numero di mine adiacenti (in orizzontale, verticale o diagonale) a quella casella. L'obiettivo del gioco è quello di scoprire tutte le caselle che non contengono una mina senza fare clic su una casella che contiene una mina.

L'implementazione del gioco del campo minato prevede la creazione di una griglia di celle, alcune delle quali contengono una mina. L'utente può cliccare su una cella per scoprirne il contenuto. Se la cella contiene una mina, il gioco è perso. Se la cella non contiene una mina, viene mostrato il numero di mine adiacenti alla cella.

Inoltre, è possibile contrassegnare le celle che l'utente ritiene contengano una mina con una bandierina, utilizzando il tasto destro del mouse. Se tutte le mine sono state contrassegnate correttamente, l'utente vince il gioco.

Di seguito il codice per la gestione del gioco del "campo minato":

```
import React, { useState, useEffect } from 'react';
import './App.css';


function Minefield() {
  const [board, setBoard] = useState([]);
  const [gameOver, setGameOver] = useState(false);
  const [gameWon, setGameWon] = useState(false);
  const [mineCount, setMineCount] = useState(0);
  const [flagCount, setFlagCount] = useState(0);

  const createBoard = (rows, cols, mines) => {
    const board = Array(rows)
      .fill(null)
      .map(() =>
        Array(cols)
          .fill(null)
          .map(() => ({ mine: false, revealed: false, flag: false }))
      );
    let mineCount = 0;
    while (mineCount < mines) {
      const row = Math.floor(Math.random() * rows);
      const col = Math.floor(Math.random() * cols);
      if (!board[row][col].mine) {
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
        board[row][col].mine = true;
        mineCount++;
      }
    }
    setBoard(board);
    setMineCount(mineCount);
  };

  const handleCellClick = (row, col) => {
    if (gameOver || gameWon) {
      return;
    }
    const cell = board[row][col];
    if (cell.flag) {
      return;
    }
    if (cell.mine) {
      setGameOver(true);
    } else {
      const newBoard = [...board];
      newBoard[row][col].revealed = true;
      setBoard(newBoard);
      if (checkWin(newBoard)) {
        setGameWon(true);
      }
    }
  };

  const handleCellRightClick = (event, row, col) => {
    event.preventDefault();
    if (gameOver || gameWon) {
      return;
    }
    const cell = board[row][col];
    if (!cell.revealed) {
      const newBoard = [...board];
      newBoard[row][col].flag = !cell.flag;
      setBoard(newBoard);
      setFlagCount(flagCount + (cell.flag ? -1 : 1));
    }
  };

  const checkWin = (board) => {
    let revealedCount = 0;
    let flaggedCount = 0;
    for (let row = 0; row < board.length; row++) {
      for (let col = 0; col < board[0].length; col++) {
        if (board[row][col].revealed) {
          revealedCount++;
        } else if (board[row][col].flag) {
          flaggedCount++;
        }
      }
    }
    return revealedCount + flaggedCount === board.length * board[0].length &&
flaggedCount === mineCount;
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
  };

  useEffect(() => {
    createBoard(10, 10, 10);
  }, []);

    const renderCell = (row, col) => {
    const cell = board[row][col];
    let content;
    if (gameOver && cell.mine) {
      content = '💣';
    } else if (cell.flag) {
      content = '🚩';
    } else if (cell.revealed && cell.mine) {
      content = '💣';
    } else if (cell.revealed) {
      const adjacentMines = getAdjacentMines(row, col);
      content = adjacentMines === 0 ? null : adjacentMines;
    } else {
      content = null;
    }
    return (
      <div
        className={`cell ${cell.revealed ? 'revealed' : ''} ${cell.flag ? 'flagged' :
''}`}
        onClick={() => handleCellClick(row, col)}
        onContextMenu={(event) => handleCellRightClick(event, row, col)}
      >
        {content}
      </div>
    );
  };

  const getAdjacentMines = (row, col) => {
    let count = 0;
    for (let i = Math.max(row - 1, 0); i <= Math.min(row + 1, board.length - 1); i++) {
      for (let j = Math.max(col - 1, 0); j <= Math.min(col + 1, board[0].length - 1);
j++) {
        if (board[i][j].mine) {
          count++;
        }
      }
    }
    return count;
  };

  const resetGame = () => {
    createBoard(10, 10, 10);
    setGameOver(false);
    setGameWon(false);
    setFlagCount(0);
  };

  return (
    <div className="game">
      <h1>Campo Minato</h1>
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
      <div className="board">
        {board.map((row, rowIndex) => (
          <div key={rowIndex} className="row">
            {row.map((_, colIndex) => (
              <React.Fragment                        key={colIndex}>{renderCell(rowIndex,
colIndex)}</React.Fragment>
            ))}
          </div>
        ))}
      </div>
      {gameOver && (
        <div className="message">
          Hai perso!{' '}
          <button onClick={resetGame}>Gioca di nuovo</button>
        </div>
      )}
      {gameWon && (
        <div className="message">
          Hai vinto!{' '}
          <button onClick={resetGame}>Gioca di nuovo</button>
        </div>
      )}
      {!gameOver && !gameWon && (
        <div className="message">
          Mine rimanenti: {mineCount - flagCount}
        </div>
      )}
    </div>
  );
}

export default Minefield;
```

Il file .css associato è il seguente:

```
main {
  text-align: center;
  vertical-align: center;
  line-height: 100vh
}

.game {
  text-align: center;
}

.cell {
  border: 1px solid #999;
  display: inline-block;
  font-size: 1.5rem;
  height: 1.5em;
  line-height: 1.5em;
  text-align: center;
  vertical-align: middle;
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
  width: 1.5em;
}

.cell.flagged {
  color: blue;
}

.cell.revealed {
  background-color: #ddd;
}

.row {
  line-height: 0;
}

.message {
  margin-top: 1em;
}
```

Quando l'utente fa clic su una cella del campo minato, viene scatenato l'evento onClick del componente React che rappresenta quella cella: tale evento è gestito dalla funzione handleCellClick(row, col), che viene passata come prop al componente Cell. Questa funzione prende in input le coordinate della cella (ovvero il numero di riga e di colonna) e aggiorna lo stato del gioco in base al contenuto della cella cliccata.

In particolare, se la cella cliccata contiene una mina, il gioco è perso e viene mostrato il contenuto di tutte le celle del campo minato. Se la cella cliccata non contiene una mina, viene mostrato il numero di mine adiacenti alla cella cliccata e la cella viene contrassegnata come scoperta. Se la cella cliccata non contiene mine adiacenti, la funzione handleCellClick viene richiamata in modo ricorsivo per tutte le celle adiacenti alla cella cliccata fino a quando tutte le celle adiacenti sono state scoperte.

Una volta aggiornato lo stato del gioco, il componente React viene renderizzato nuovamente per mostrare le modifiche apportate. In questo modo, l'utente può continuare a giocare fino a quando non scopre tutte le celle non contenenti mine o fa clic su una cella contenente una mina, determinando la fine del gioco.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

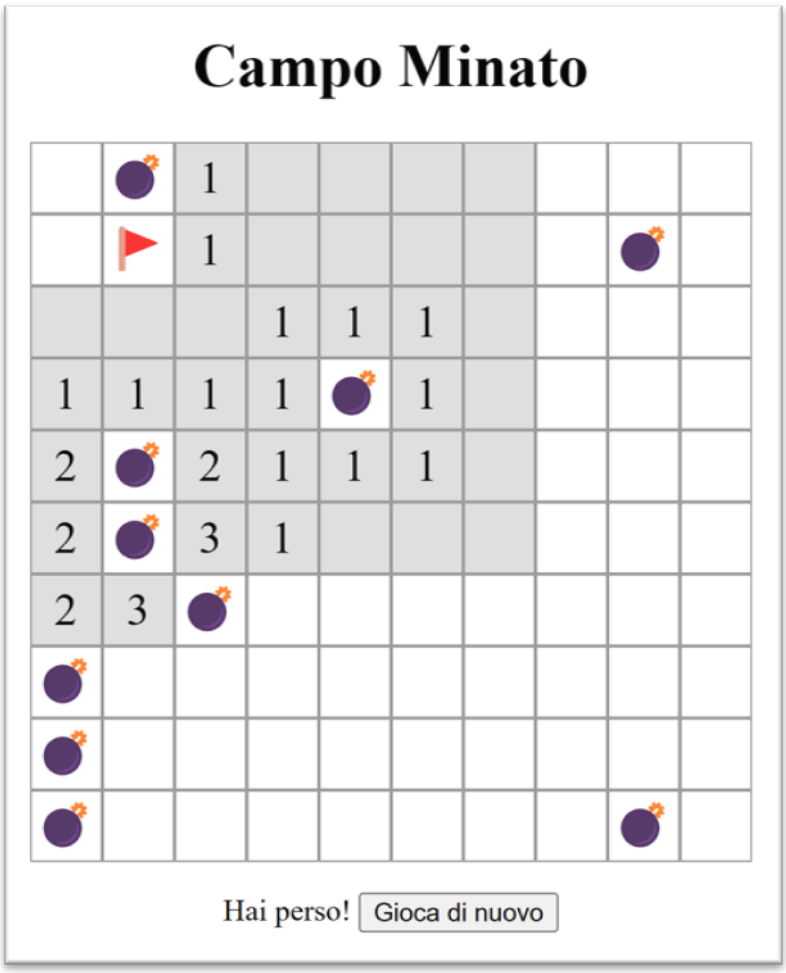


***Figura 2** - Campo Minato*

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- V. Roberto, M. Frailis, A. Gugliotta, P. Omero. Introduzione alle Tecnologie Web, McGraw-Hill, 2010.
- A. S. Tanembaum. Computer Networks (4 ed.). Prentice Hall (4 ed.), 2003.
- Marco Andreini, Patrizia Andronico, Arianna Del Soldato, Cristian Lucchesi, Maurizio Martinelli, Giuseppe Vasarelli. Manuale corso di Tecnologie Web Istituto di Informatica e Telematica del CNR, 2005.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Sitografia

- HTML 4.01 Specification: http://www.w3.org/TR/html401/
- HTML5: http://www.w3.org/TR/html5/
- Javascript: JavaScript Tutorial (w3schools.com)
- HTML: http://www.w3schools.com/html/default.asp
- CSS: https://www.w3schools.com/css/default.asp
- CSS: http://www.w3.org/TR/CSS/
- NODEJS: https://nodejs.org
- REACT: https://it.legacy.reactjs.org/

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).